

Базовые сущности Kubernetes: Job, CronJob, ConfigMap, Secret

Microservice Architecture

REC **Проверить, идет ли запись**

Меня хорошо видно & слышно?





Майоров Антон

Старший разработчик ПО

- 10+ лет опыта разработки ПО на платформе .NET
- Специализация: приложения со сложной бизнес логикой
- Последние 4 года - старший системный разработчик в Ренессанс Страхование

@amaiorov

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе

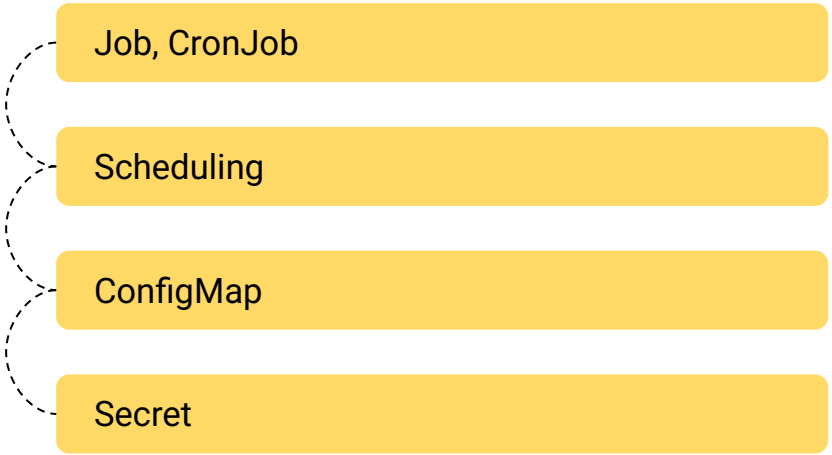


Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара



Цели вебинара

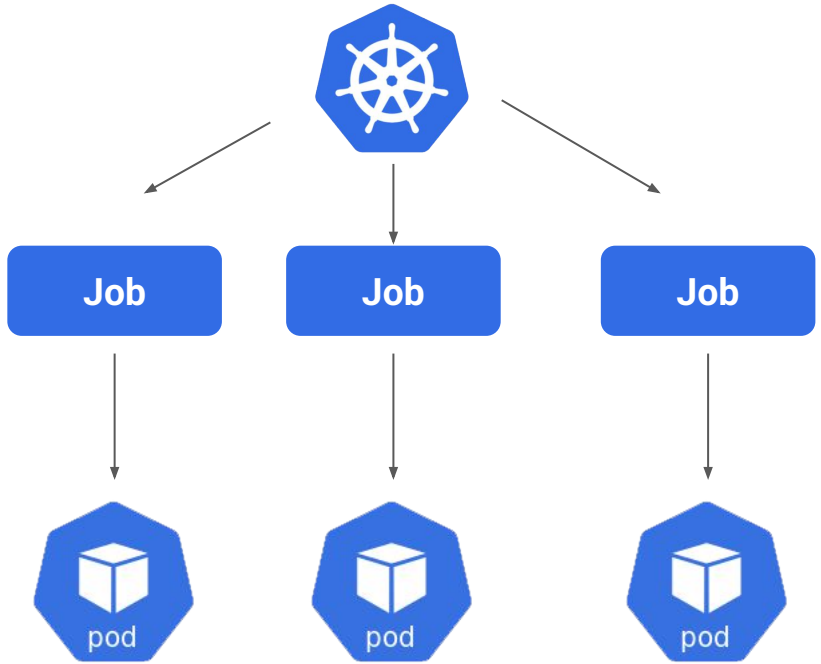
К концу занятия вы сможете

1. Понять особенности функционирования Job и CronJob
2. Понимать механизмы nodeSelector, Affinity и Tolerations
3. Создавать и использовать объекты k8s ConfigMap & Secret

Job. CronJob.

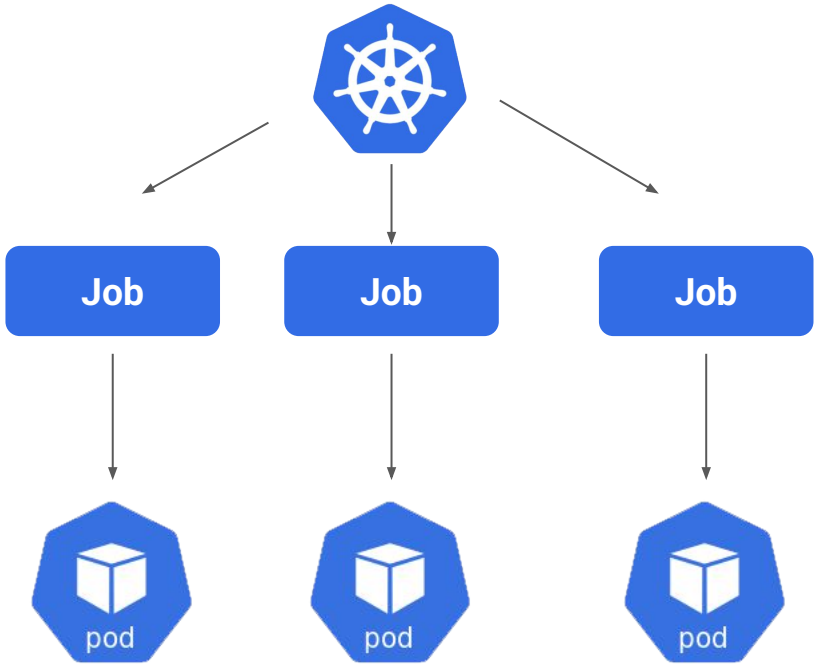
Job. Зачем

- Выполнение конечных задач
- Освобождение ресурсов, когда задача не выполняется



Job. Use cases

- Миграции БД
- Data-processing
- Выполнение ресурсоемких задач
- Технические/рутинные задачи



Job. Пример

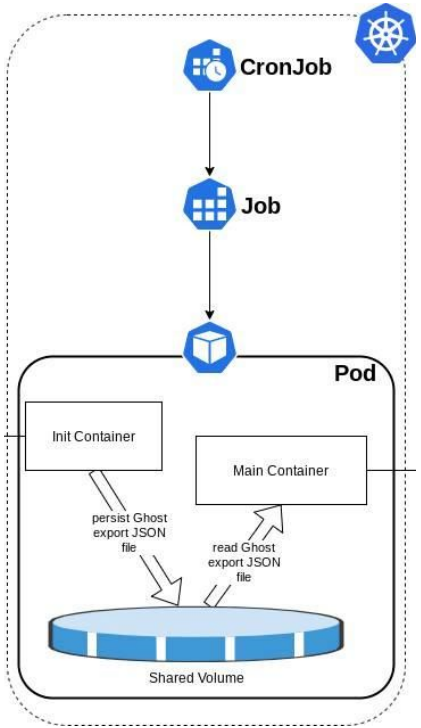
```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  template:
    spec:
      containers:
      - name: pi
        image: perl:5.34.0
        command: ["perl",  "-Mbignum=bpi", "-wle", "print bpi(2000)"]
      restartPolicy: Never
  backoffLimit: 4
```

Job. Основные параметры

- BackOffLimit
- RestartPolicy
- Completions
- Parallelism
- activeDeadlineSeconds
- ttl

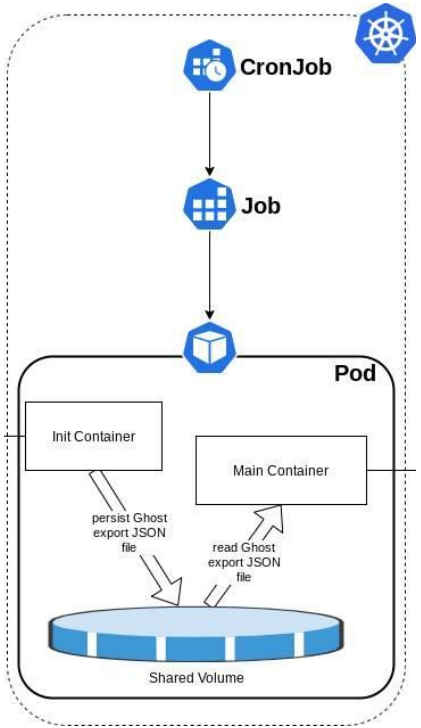
CronJob. Зачем

- Запуск в определенное время
- Периодический запуск по расписанию



CronJob. UseCases

- Пакетная обработка
- Рассылки
- Генерация отчетности
- ...



CronJob. Пример

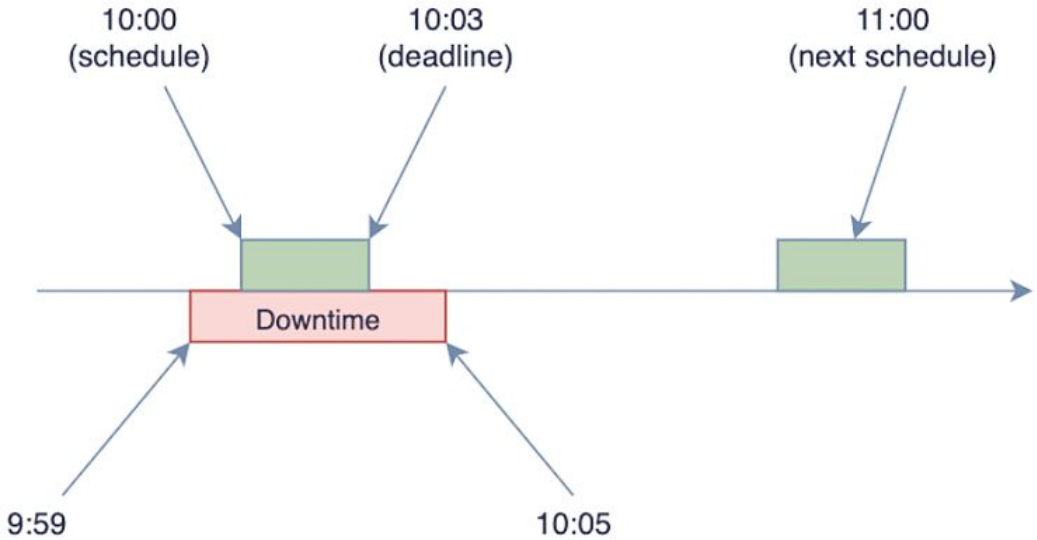
```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "* * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: hello
            image: busybox:1.28
            imagePullPolicy: IfNotPresent
            command:
            - /bin/sh
            - -c
            - date; echo Hello from the Kubernetes cluster
          restartPolicy: OnFailure
```

CronJob. Основные параметры

- schedule
- concurrencyPolicy
- startingDeadlineSeconds
- suspend

CronJob. StartingDeadlineSeconds

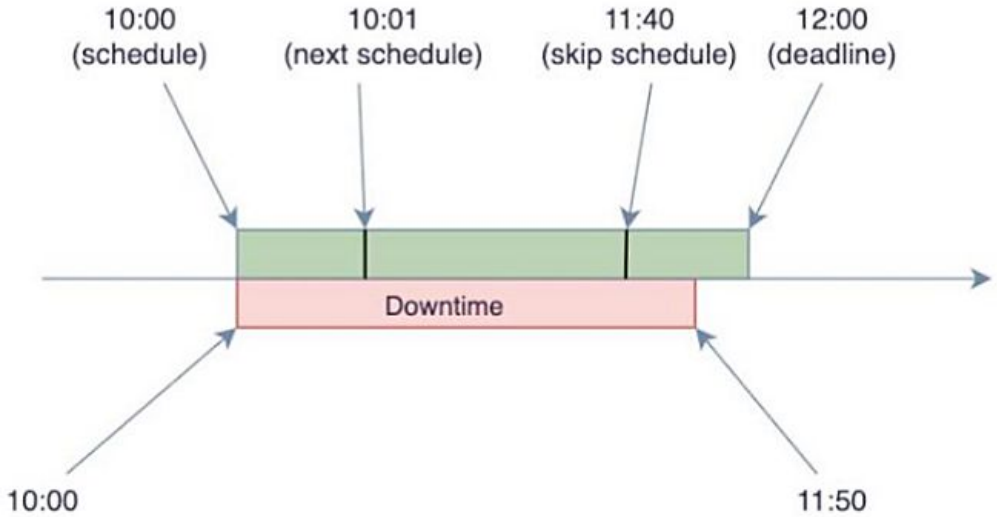
```
startingDeadlineSeconds: 180
schedule: "0 * * * *"
```



CronJob. StartingDeadlineSeconds

```
startingDeadlineSeconds: 7200
schedule: "*/1 * * * *"
```

```
Cannot determine if job needs to be
started. Too many missed start time
(> 100). Set or decrease
.spec.startingDeadlineSeconds or
check clock skew.
```



Вопросы?

+ если есть вопросы

− если вопросов нет

Управление шедулингом

nodeSelector

- Присваиваем хосту метку

```
kubectl label node fast-node kubernetes.io/disk=ssd
```

- Указываем метку в спецификации пода

```yaml
apiVersion: v1
kind: Pod
metadata:
...
spec:
  nodeSelector:
    kubernetes.io/disk: ssd
```

Affinity и anti-affinity

- Более удобный способ описания правил для Scheduler
- Возможность указывать hard и soft правила
- Поддержка affinity на уровне pod
- Возможность использовать операторы
  - In & NotIn
  - Exist & DoesNotExist
  - Lt & Gt

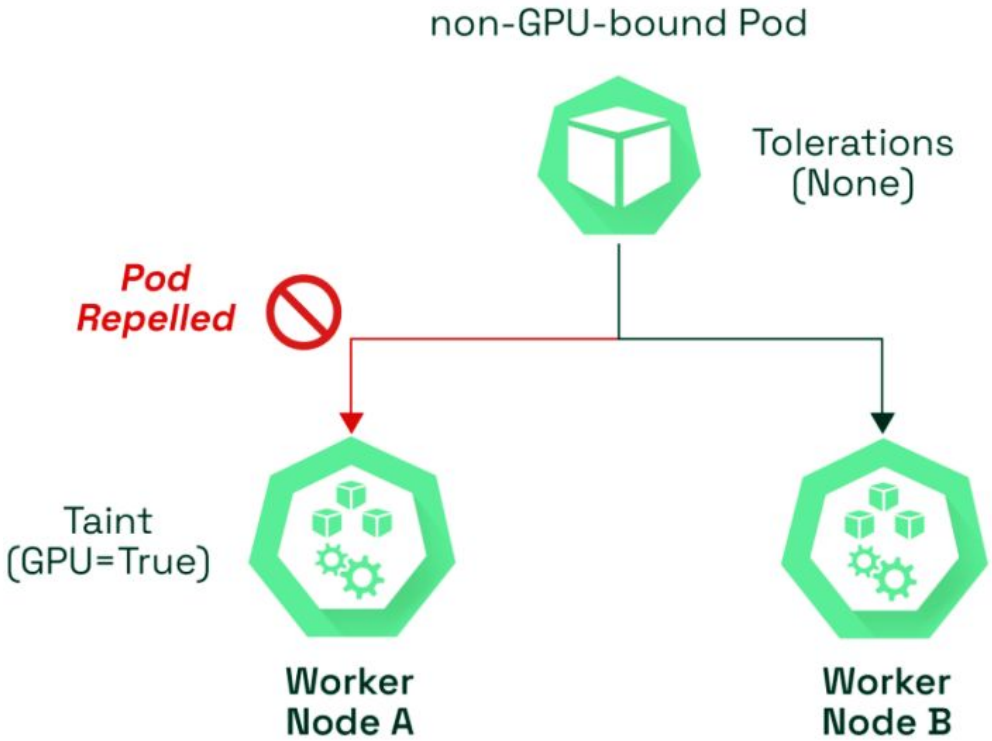
NodeAffinity

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: hard-node-affinity
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: node-restriction.kubernetes.io/disk
            operator: In
            values:
            - ssd
```

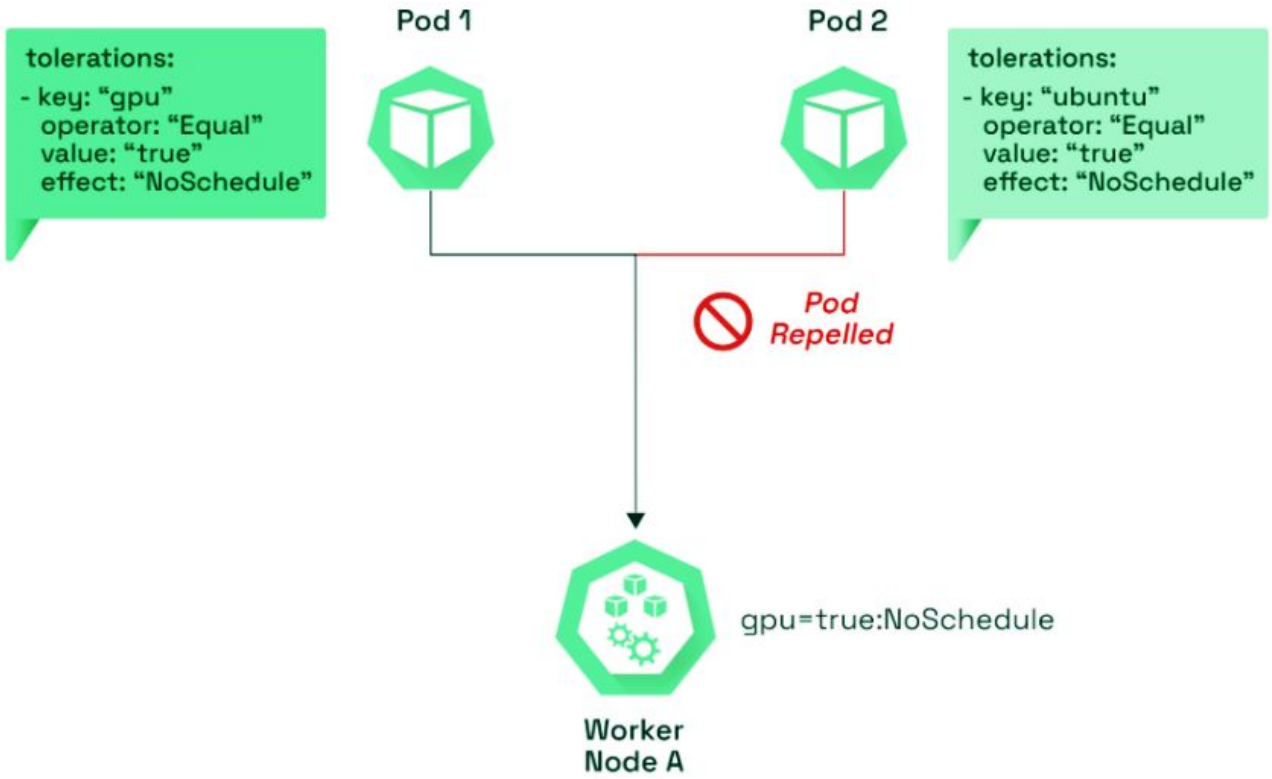
InterPodAffinity

```yaml
apiVersion: apps/v1
kind: Deployment
...
spec:
...
  replicas: 3
  template:
    metadata:
      labels:
        app: nginx
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:
                - nginx
            topologyKey: "kubernetes.io/hostname"
```

Taints


non-GPU-bound Pod
Tolerations
(None)
Pod
Repelled
Taint
(GPU=True)
Worker
Node A
Worker
Node B

Tolerations



Вопросы?

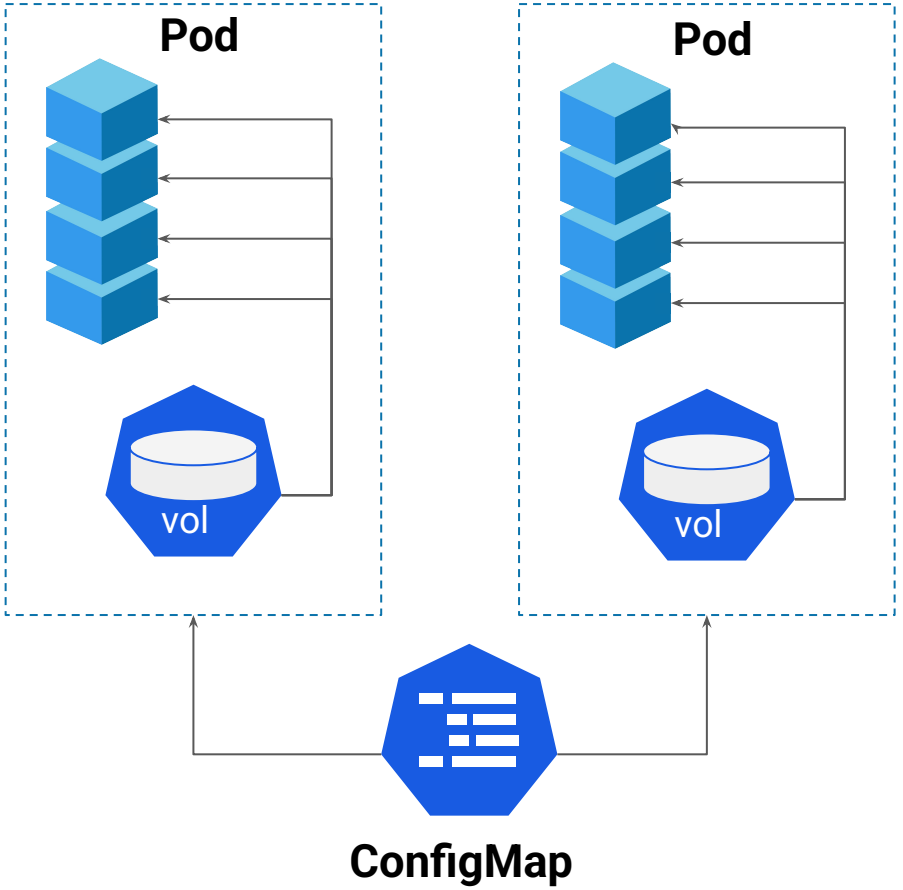
+ если есть вопросы

− если вопросов нет

ConfigMap

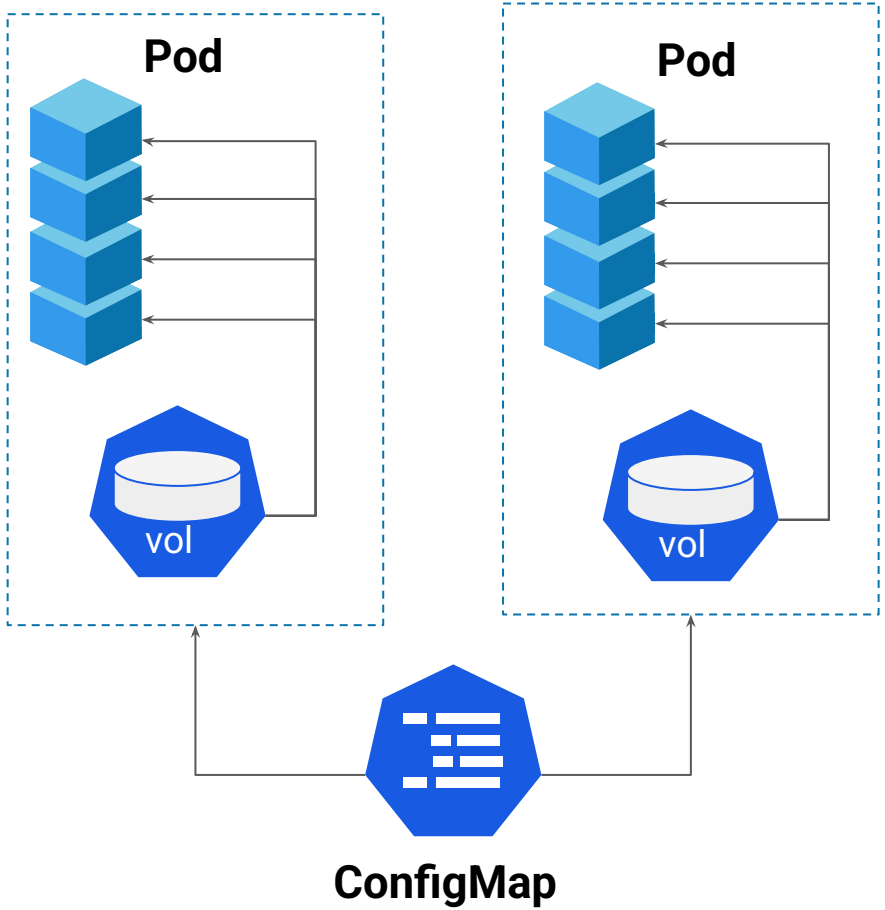
ConfigMap. Зачем

- Хранение параметров в формате ключ: значение
- Только не конфиденциальные параметры



ConfigMap. Использование

- Переменные окружения
- Монтирование в под



ConfigMap. Пример

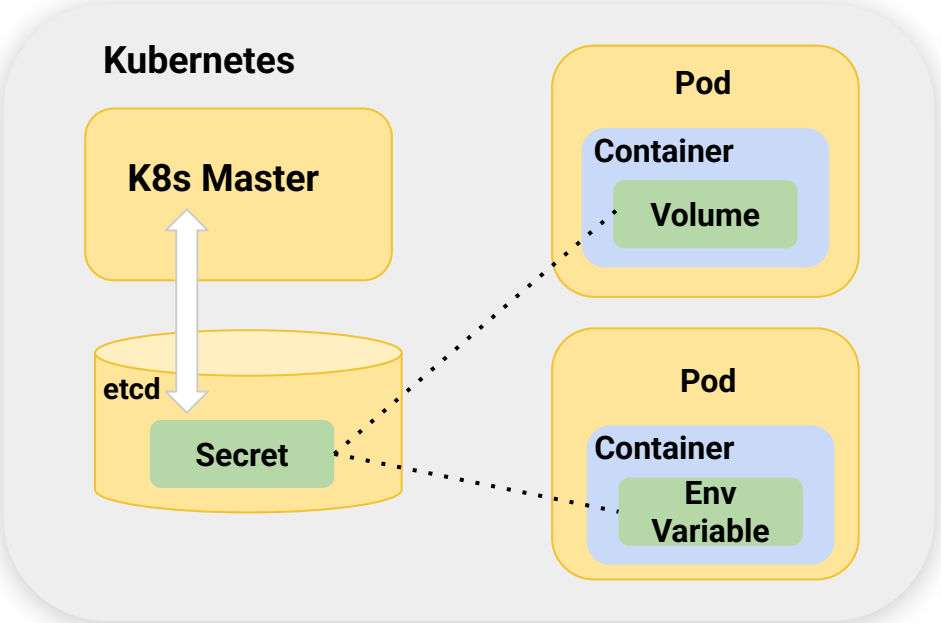
```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  # property-like keys; each key maps to a simple value
  player_initial_lives: "3"
  ui_properties_file_name: "user-interface.properties"

  # file-like keys
  game.properties: |
    enemy.types=aliens,monsters
    player.maximum-lives=5
  user-interface.properties: |
    color.good=purple
    color.bad=yellow
    allow.textmode=true
```

Secret

Secret. Зачем

- Хранение параметров в формате ключ:значение
- Для хранения конфиденциальных и чувствительных параметров



Secret. Пример

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: dotfile-secret
data:
  .secret-file: dmFsdWUtMg0KDQo=
```

Вопросы?

+ если есть вопросы

− если вопросов нет

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу ✏️❤️